author: Harzarul

> bootcamp.learn()

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 21

Async JavaScript and browser requests

learn / type / run / explain

Plain meaning and technical view

PLAIN MEANING

Network work finishes later and may fail even when the interface is working.

TECHNICAL VIEW

Promises and await handle asynchronous results. fetch requires explicit status checks. FormData handles files. AbortController cancels obsolete work.

REAL-WORLD USE

Read posts, submit a form and preserve user input when the API rejects it.

QUICK TIP

A rejected HTTP status does not automatically reject fetch. Use response.ok. Let the browser set multipart Content-Type.

INSTALL / RUN

Browser main route: start the local training API and HTTP server as shown next. No npm package. Save, reload, click Run, then read DevTools Console.

Two files, one lesson

DO THIS / WHY

Create browser-practice beside your Python practice folder. Save the HTML launcher and JS source together. The HTML supplies the button/status element; the JS holds the four examples and your two exercises. Use DevTools Console for console.log output.

CHECKPOINT / TIP

This is supporting setup; the four worked examples follow next.

Local training API / supporting file

DO THIS / WHY

Create training_api.py in your Python practice folder. Type the supporting server code shown next. It uses in-memory test records and deliberately fake identity; never deploy it or use real data.

CHECKPOINT / TIP

Restart this server to reset the records. Real Supabase verification is in Postlight.

Local training API / supporting file (1/5)

training_api.py

+ = same line

```
01    """Local-only teaching API for JS/React labs. Fake data and fake identity.
02    Run with uvicorn on 127.0.0.1:8765. Never deploy this or use real data.
03    Restart the process to reset records. Real app verifies Supabase tokens.
04    """
05    from typing import Annotated
06    from fastapi import FastAPI, Form, File, UploadFile, Header, HTTPException, Response
07    from fastapi.middleware.cors import CORSMiddleware
08
09    app = FastAPI(title="Local classroom fake API")
10    app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173",
+     "http://127.0.0.1:5173", "http://localhost:8080"],
11                       allow_methods=["GET", "POST", "DELETE"], allow_headers=["Authorization",
```

PRO TIP Restart this server to reset the records. Real Supabase verification is in Postlight.

Local training API / supporting file (2/5)

training_api.py

+ = same line

```
+       "Content-Type"])
12      posts = {1: {"id": 1, "title": "Training story", "body": "A local test record", "author_id":
+       "ada"}}
13      next_id = 2
14
15      @app.get("/posts")
16      def list_posts():
17          return list(posts.values())
18
19      @app.get("/posts/{post_id}")
20      def detail(post_id: int):
21          if post_id not in posts:
```

PRO TIP Restart this server to reset the records. Real Supabase verification is in Postlight.

Local training API / supporting file (3/5)

training_api.py

+ = same line

```
22          raise HTTPException(404, "Post not found")
23      return posts[post_id]
24
25  @app.post("/posts", status_code=201)
26  async def create(title: Annotated[str, Form()], body: Annotated[str, Form()],
27                   image: UploadFile | None = File(default=None),
28                   authorization: str | None = Header(default=None)):
29      global next_id
30      if not title.strip() or not body.strip():
31          raise HTTPException(422, "Title and body required")
32      # Plain JS create is anonymous in this lab. /protected-posts below is fake-protected.
33      if image:
```

PRO TIP Restart this server to reset the records. Real Supabase verification is in Postlight.

Local training API / supporting file (4/5)

training_api.py

+ = same line

```
34          await image.read(5_000_001)  # Demonstrates multipart, not production validation.
35      post = {"id": next_id, "title": title.strip(), "body": body.strip(), "author_id": "ada"}
36      posts[next_id] = post
37      next_id += 1
38      return post
39
40  @app.post("/protected-posts", status_code=201)
41  async def protected_create(title: Annotated[str, Form()], body: Annotated[str, Form()],
42                             image: UploadFile | None = File(default=None),
43                             authorization: str | None = Header(default=None)):
44      if authorization != "Bearer classroom-fake-token":
45          raise HTTPException(401, "Fake sign in required")
```

PRO TIP Restart this server to reset the records. Real Supabase verification is in Postlight.

Local training API / supporting file (5/5)

training_api.py

+ = same line

```
46        return await create(title, body, image, authorization)
47
48    @app.delete("/posts/{post_id}", status_code=204)
49    def delete(post_id: int):
50        if posts.pop(post_id, None) is None:
51            raise HTTPException(404, "Post not found")
52        return Response(status_code=204)
53
```

PRO TIP Restart this server to reset the records. Real Supabase verification is in Postlight.

Start API / Python practice folder

DO THIS / WHY

Use the practice venv from chapter 11. Keep this terminal running. Check http://127.0.0.1:8765/posts: JSON includes Training story.

CHECKPOINT / TIP

Use the named folder and your OS command. Keep long-running servers in separate terminals.

Start API / Python practice folder / terminal

terminal

Windows

```
> .\.venv\Scripts\python.exe -m pip install fastapi uvicorn python-multipart
```

macOS

```
> ./.venv/bin/python -m pip install fastapi uvicorn python-multipart
```

Windows

```
> .\.venv\Scripts\python.exe -m uvicorn training_api:app --host 127.0.0.1 --port 8765
```

macOS

```
> ./.venv/bin/python -m uvicorn training_api:app --host 127.0.0.1 --port 8765
```

Type commands only. Omit the > prompt and OS labels.

Browser launcher / supporting file

DO THIS / WHY

Save this HTML next to chapter_21_async_javascript_and_browser_requests.js. Type shared setup once, then wrap each JS example in { ... } so repeated names do not collide. Imports stay outside braces. Leave exercises blank until the four examples are complete.

CHECKPOINT / TIP

Modules are cached. Save the JS and reload this page before another run.

Browser launcher / supporting file (1/2)

chapter_21_browser_harness.html

+ = same line

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>JomCode browser requests</title></head>
<body>
  <h1>Async requests</h1>
  <p>Open DevTools Console. Save JS, then reload to run again.</p>
  <button id="run">Run examples and solutions</button>
  <p id="output" role="status"></p>
  <script type="module">
    const button = document.querySelector("#run");
    button.addEventListener("click", async () => {
```

PRO TIP Modules are cached. Save the JS and reload this page before another run.

Browser launcher / supporting file (2/2)

chapter_21_browser_harness.html

+ = same line

```
12          button.disabled = true;
13          try {
14            await import("./chapter_21_async_javascript_and_browser_requests.js");
15          } catch (error) {
16            document.querySelector("#output").textContent = error.message;
17          }
18        });
19      </script>
20    </body>
21    </html>
```

PRO TIP Modules are cached. Save the JS and reload this page before another run.

Serve the browser-practice folder

DO THIS / WHY

New terminal in browser-practice, not the API folder. Open http://localhost:8080/chapter_21_browser_harness.html. Open DevTools > Console, click Run once. Expected: titles, 404, 204 and cancelled. Exercise 1 later updates the status paragraph.

CHECKPOINT / TIP

Use the named folder and your OS command. Keep long-running servers in separate terminals.

Serve the browser-practice folder / terminal

terminal

Windows

```
> py -m http.server 8080
```

macOS

```
> python3 -m http.server 8080
```

Type commands only. Omit the > prompt and OS labels.

Shared setup

chapter_21_async_javascript_and_browser_requests.js

+ = same line

```
01  const API = "http://127.0.0.1:8765";
02  // Local classroom fake only. Keep this before all example blocks.
```

PRO TIP Keep these imports, definitions and fixtures when running the examples.

Worked example 1: Await the result

chapter_21_async_javascript_and_browser_requests.js

+ = same line

```
01  async function readTitles() {
02    const response = await fetch(API + "/posts");
03    if (!response.ok) throw new Error("Load failed");
04    const posts = await response.json();
05    return posts.map((p) => p.title);
06  }
07  console.log(await readTitles());
```

PRO TIP await works in an async function or a module; HTTP work can fail.

Run and explain example 1

EXPECTED OUTPUT

A list containing Training story (unless lab data was changed).

WHY IT BEHAVES THIS WAY

A Promise. await pauses this async operation while other work can continue.

Worked example 2: HTTP error handling

chapter_21_async_javascript_and_browser_requests.js

+ = same line

```
01  const response = await fetch(API + "/posts/999");
02  if (!response.ok) {
03    const problem = await response.json();
04    console.log(response.status, problem.detail);
05  }
```

PRO TIP fetch does not throw for 404; check response.ok yourself.

Run and explain example 2

EXPECTED OUTPUT

404 Post not found

WHY IT BEHAVES THIS WAY

No. A network failure can reject the Promise. HTTP failure requires checking status or ok.

Worked example 3: Multipart and empty delete

chapter_21_async_javascript_and_browser_requests.js

+ = same line

```
01  const form = new FormData();
02  form.set("title", "Temporary");
03  form.set("body", "A lab post");
04  form.set("image", new Blob(["demo bytes"]), "demo.txt");
05  const created = await fetch(API + "/posts", { method: "POST", body: form });
06  if (!created.ok) throw new Error("Create failed");
07  const post = await created.json();
08  const removed = await fetch(API + "/posts/" + post.id, { method: "DELETE" });
09  if (!removed.ok) throw new Error("Delete failed");
10  console.log(removed.status, removed.status === 204 ? "no body" : await removed.text());
```

PRO TIP A 204 response has no JSON. Let FormData set Content-Type.

Run and explain example 3

EXPECTED OUTPUT

204 no body. Lab accepts demo bytes. Real guided app verifies actual images.

WHY IT BEHAVES THIS WAY

The browser supplies the multipart boundary. A 204 response contains no JSON.

Worked example 4: Cancel obsolete work

chapter_21_async_javascript_and_browser_requests.js

+ = same line

```
01  const controller = new AbortController();
02  controller.abort();
03  try {
04    await fetch(API + "/posts", { signal: controller.signal });
05  } catch (error) {
06    if (error instanceof Error && error.name === "AbortError") console.log("cancelled");
07    else throw error;
08  }
```

PRO TIP Ignore intentional AbortError; report other network failures.

Run and explain example 4

EXPECTED OUTPUT

cancelled

WHY IT BEHAVES THIS WAY

Usually no. React cleanup can cancel requests whose component no longer needs the result.

Exercise 1: Loader and DOM status

YOUR TASK

Write a loader with status checking. In the browser display loaded count using textContent, not innerHTML. Node may log instead.

CHECK THESE CASES

Valid route shows count. Missing route throws a useful error. No unsafe HTML insertion.

HINT AFTER A TRY

The harness script uses type=module. DOM lookup requires the element to exist.

Exercise 2: Failure-preserving form

YOUR TASK

Submit blank title and keep the draft object unchanged if the server rejects it. Report the failure.

CHECK THESE CASES

422 returned. Draft still contains typed body. No success message.

HINT AFTER A TRY

Clear a form only after a successful response.

Checkpoint before moving on

SHOW AND EXPLAIN

Run and explain all four examples. Complete both exercises and verify every listed acceptance case.

TRANSFER TO POSTLIGHT

Read posts, submit a form and preserve user input when the API rejects it.